TripWeaver AI

Complete Project Report

A production-oriented, multi-agent travel-planning application

TripWeaver AI (shown in the browser as TripMate AI) turns a natural-language travel request into a research-backed, budget-aware travel itinerary. It combines FastAPI, LangGraph, Groq, PostgreSQL and Model Context Protocol (MCP) integrations, then deliberately pauses for a human review before producing the final plan.

Report scope	Repository implementation, architecture, workflows, APIs, configuration, testing, deployment and technical findings
Repository	TripWeaver-AI-Multi-Agent-Travel-Orchestration
Application version	FastAPI metadata declares version 2.0.0
Prepared	19 October 2026
License	MIT

Executive summary

The project is a thoughtful implementation of supervisor-led agent orchestration. A routing agent first checks whether the input is appropriate for travel planning, selects only relevant specialists, captures trip constraints, and directs execution through the selected agents. Flight, hotel, weather and budget evidence is consolidated by an itinerary agent. LangGraph interruption provides a clean human-in-the-loop checkpoint, while PostgreSQL checkpoints preserve the conversation state across requests and browser sessions.

The system is strong as a planning assistant, not a booking engine. Its most important operational requirements are valid API credentials, a reachable PostgreSQL database and local availability of the uvx executable for AviationStack integration. Two code-level issues should be prioritized before production rollout: the configured weather-server filename does not match the file in the repository, and the Docker image does not install uv/uvx.

1. Product scope and capabilities

The application accepts free-form prompts such as a multi-day international trip with an origin, budget, hotels and sightseeing. It dynamically chooses specialist work rather than always invoking every agent.

Capability	Implementation	User value
Supervisor routing	LLM extracts constraints and selects agents from a controlled registry; itinerary is always included.	Avoids unnecessary tool calls and focuses work on the request.
Input guardrail	LLM JSON decision blocks clearly non-travel or harmful requests; parser failures fail open to preserve planning availability.	Keeps the product in scope while avoiding brittle failures.
Live research	AviationStack, Tavily and OpenWeather are accessed through MCP clients/servers.	Adds external flight context, hotel research and weather conditions.
Budget analysis	Dedicated LLM specialist estimates categories, risks and savings strategies.	Makes feasibility visible even when prices are approximate.
Human review	LangGraph interrupt pauses after the draft; approve or revise resumes the same thread.	Keeps the traveler in control before finalization.
Persistent sessions	PostgreSQL PostgresSaver plus browser localStorage thread ID.	Supports multi-turn continuity across browser interactions.
Result delivery	Markdown is rendered in a responsive vanilla-JS interface with copy and browser-side PDF export.	Easy reading and sharing of an itinerary.

2. Architecture

The system is organized in four layers: browser UI, HTTP API, graph orchestration, and external/state services. The application code is compact but maintains clear responsibility boundaries.

Browser	FastAPI	LangGraph workflow	Services & persistence
HTML/CSS/JS marked.js html2pdf.js localStorage	Pydantic validation /api/travel /api/travel/approve asyncio.to_thread	Supervisor & guardrail Specialist agents HITL interrupt Final response	Groq LLM Tavily MCP AviationStack MCP Weather MCP PostgreSQL

Request lifecycle

1. The browser submits a prompt and its stored thread ID. 2. FastAPI validates the input and runs the synchronous graph function in a worker thread. 3. The supervisor guards, interprets and routes the request. 4. Selected specialists gather or synthesize evidence. 5. The itinerary agent prepares a draft. 6. LangGraph interrupts for human approval. 7. A second API call resumes the exact checkpoint and the final agent produces polished Markdown.

3. Agent orchestration

The graph uses a typed TravelState that accumulates messages, constraints, selected agents, specialist results, approval metadata and an LLM-call counter. Agent selection follows a fixed ordering to keep routing deterministic after the supervisor has selected the set.

Node	Primary responsibility	Inputs / outputs
supervisor	Travel guardrail, constraint extraction and agent selection.	Prompt -> allowed flag, rationale, selected agents, structured constraints.

Node	Primary responsibility	Inputs / outputs
flight_agent	Airport/airline route guidance.	AviationStack list_airports/list_airlines -> LLM flight guidance.
hotel_agent	Accommodation research.	Tavily search -> hotel/neighborhood evidence or graceful fallback.
weather_agent	Current conditions and forecast context.	LLM destination extraction + local Weather MCP -> weather text.
budget_agent	Feasibility and cost risk assessment.	Constraints and prior evidence -> cost categories/savings advice.
itinerary_agent	Draft integrated plan.	Selected results -> practical review-ready itinerary.
human_approval	Explicit review checkpoint.	interrupt payload -> approved flag and feedback on resume.
final_agent	Polished traveler-facing result.	Draft + approval/feedback -> structured Markdown final response.

4. Data, integrations and resilience

MCP acts as the integration boundary. The MultiServerMCPClient contains three named servers: Tavily over streamable HTTP, AviationStack via a uvx subprocess, and the local weather server via Python stdio. Each tool is loaded by server and tool name, limiting the impact of an unavailable integration.

Integration	Purpose	Failure behavior
Groq / ChatGroq	Supervisor decisions, specialist synthesis and final response.	Startup validates GROQ_API_KEY; runtime errors propagate to API 500.
Tavily MCP	Hotel and accommodation research.	Hotel agent returns clearly labeled non-live fallback guidance.
AviationStack MCP	Airports and airlines context.	Flight agent returns an unavailable message when it fails.
OpenWeather / FastMCP	Current weather and first five forecast records in metric units.	Weather agent provides seasonal-advice fallback after destination extraction.
PostgreSQL	LangGraph checkpointer for resumable workflows.	Connection is created at import/startup; DB problems prevent normal operation.

State persistence

The backend adds sslmode=require to DATABASE_URL when the URL does not already specify an SSL mode. A PostgresSaver is initialized and passed to graph.compile(). The generated or supplied thread_id is used as LangGraph configurable state, while the browser stores it as travel_thread_id in localStorage. This pairing allows a review action to continue an interrupted plan rather than starting over.

5. API contract

Endpoint	Purpose	Validation / result
GET /	Serves the single-page TripMate UI.	Jinja2 renders templates/index.html.
GET /health	Basic health endpoint.	Returns status, message and advertised key features.
POST /api/travel	Starts a planning graph run.	Requires nonblank message; optional thread_id. Returns serialized state and requires_approval after the interrupt.
POST /api/travel/approve	Resumes an interrupted graph.	Requires nonempty thread_id and approved boolean. Rejections must include nonblank feedback.
GET /favicon.ico	Minimal fallback endpoint.	Returns an empty JSON object.

6. Frontend behavior

The client is deliberately lightweight: no frontend framework or build chain is required. The script sends JSON using fetch(), displays server errors, renders Markdown using marked.js, retains the thread identifier locally, copies text to the clipboard, and uses html2pdf.js to generate an A4 PDF from the result content. The styling establishes a responsive dark visual theme with gradients and quick-prompt chips.

7. Installation and deployment

The project targets Python 3.11+ and provides pinned dependencies. A local PostgreSQL-compatible DATABASE_URL plus Groq credentials are essential. Tavily, AviationStack and OpenWeather credentials enable their respective research capabilities. uv is required because AviationStack runs through uvx.

Area	Expected configuration
Core environment	GROQ_API_KEY and DATABASE_URL are required; GROQ_MODEL is optional and defaults to openai/gpt-oss-120b.
Specialists	TAVILY_API_KEY, AVIATIONSTACK_API_KEY (or AVIATION_STACK_API_KEY), and OPENWEATHER_API_KEY.
Observability	LANGSMITH_TRACING, LANGSMITH_API_KEY and LANGSMITH_PROJECT are optional.
Local run	Activate .venv, install requirements, then run uvicorn app:app --reload.
Container run	Dockerfile exposes 8000 and launches uvicorn on 0.0.0.0:8000; environment values are passed through .env.

8. Quality assurance

The repository documents 177 mocked tests across three suites. The test design avoids paid/external calls and database connections while checking route validation, model behavior, workflow helpers, routing, MCP utility behavior, weather error handling and project files. A syntax-only compilation command is also provided for the four Python application modules.

Suite	Documented count	Coverage
test_backend_units.py	62	Helpers, conditional routing, supervisor/guardrail behavior, serialization, registry and DB URL logic.
test_weather_and_utils.py	53	Weather MCP API/key/error cases, MCP-client helpers and repository structure.
test_api.py	62	Endpoints, Pydantic models, validation, static assets and edge cases.

9. Findings and recommendations

The following items are based on direct repository inspection and are ordered by operational impact.

Priority	Finding	Recommended action
High	mcp_client.py configures WEATHER_SERVER_PATH as custom_weather_mcp_server.py, but the repository contains custom_weather_mcp.py.	Change the path or rename the file; add an integration smoke test that starts the weather subprocess.
High	The Dockerfile installs build tools, git and curl but not uv. AviationStack requires uvx.	Install uv in the image and verify uvx --version during image build, or replace subprocess tooling with an included dependency.
Medium	The backend instantiates DB/LLM clients during import, making boot dependency failures immediate.	Consider lifespan initialization plus readiness reporting for clearer diagnostics and test isolation.
Medium	The UI currently presents the draft result but does not expose approve/revise controls despite the backend endpoint.	Add an approval panel with feedback input and call /api/travel/approve to complete the HITL experience.
Medium	Flight context is airport/airline data rather than confirmed ticket inventory, and live research can be unavailable.	Prominently label estimates and freshness; add provider timestamps and citations in generated plans.
Low	The report's documented test count was not re-executed as part of this report generation.	Run the documented pytest command in CI and publish results/badges on every change.

10. Project structure

The codebase is small and approachable. The core implementation lives at the repository root, while static web assets and tests are separated by function.

```
TripWeaver-AI-Multi-Agent-Travel-Orchestration/
app.py FastAPI routes, validation and static/template setup
backend.py LangGraph state, agents, routing, checkpoints and helpers
mcp_client.py Tavily, AviationStack and Weather MCP client configuration
custom_weather_mcp.py Local FastMCP OpenWeather tools over stdio
requirements.txt Pinned Python dependencies
Dockerfile Python 3.11 container image
templates/index.html Single-page application shell
static/script.js API client, Markdown render, local state and PDF download
static/style.css Responsive visual design
tests/ Backend, API, MCP and structural test coverage
```

Conclusion

TripWeaver AI has a sound product shape: it combines targeted agent specialization with explicit supervision, live-context integrations and a resumable human review point. Its architecture is suitable for iterative hardening. Correcting the weather path, containerizing uv support and completing the approval controls in the browser would materially improve end-to-end reliability. With those changes and ongoing integration tests, it can serve as a strong reference project for multi-agent travel orchestration.

Prepared from the repository contents available on 19 October 2026. No API keys or other secret values were included in this report.